

CSCE 638 NATURAL LANGUAGE PROCESSING · MODULE 03 · LECTURE NOTES

Language Models

Predicting the next token, and why that is enough.

MODULE OUTCOMES

- Define a language model and state what it is for.
- Explain n-gram models, sparsity, and smoothing.
- Explain perplexity and its limitations.
- Explain why the neural formulation generalises.
- Build and evaluate both kinds.

1 The object

$$P(w_1 \dots w_n) = \prod_i P(w_i \mid w_1 \dots w_{i-1})$$

- **The chain rule factorisation is exact.** Any distribution over sequences can be written this way, so **nothing has been approximated yet**.
- **The n-gram approximation** replaces the full history with the last k tokens — a Markov assumption that is **plainly false about language and computationally tractable**, which was the right trade for decades.
- **The neural formulation** computes $P(w_i \mid \text{context})$ as a softmax over a learned function of the whole context, **so no independence assumption is required at all** — the limit becomes architectural rather than statistical.
- **Which is the whole difference between the two families: the factorisation is shared and the amount of context conditioned upon is not**, and every advance from Module 05 onward is about conditioning on more of it more effectively.

A LANGUAGE MODEL IS A MODEL OF THE DATA, AND IT TURNS OUT TO BE A MODEL OF THE TASK

Nothing about predicting the next token mentions translation, summarisation, or question answering — and yet **a sufficiently good next-token predictor can do all three**, because each of them can be phrased as a continuation of some prefix.

Which is the observation the entire field was built on. 'Translate to French: hello →' has a **correct continuation**, so a model that predicts continuations accurately translates — without ever having been trained to translate, which was a genuine surprise when it was first demonstrated at scale.

And the training signal is free (Module 01 §3) — **any text at all is a labelled example of 'what comes next'** — which is why these models scaled when supervised approaches could not, since supervision was the binding constraint.

But it is worth being precise about what follows from this: the model is optimised for *plausible continuation*, not for *correctness* — and **that distinction is Module 12 §3's entire subject** and the origin of the confident-fabrication failure mode. A fluent false answer is a *successful* continuation by the training objective.

2 Counting, and its limit

$$P(w \mid \text{context}) = \text{count}(\text{context}, w) / \text{count}(\text{context})$$

AND MOST COUNTS ARE ZERO

a 20,000-word vocabulary gives 8×10^{12} possible trigrams. No corpus covers them. So an unseen trigram gets probability 0, and therefore the whole sequence gets probability 0.

SMOOTHING, in increasing sophistication

add-one	simple, and badly biased
back-off	use the shorter context when the longer one is unseen
interpolation	mix all orders together with learned weights
Kneser-Ney	back off by how many DISTINCT contexts a word appears in, not by its raw frequency

KNESER-NEY'S INSIGHT: "Francisco" is frequent but appears almost only after "San", so it is a poor back-off candidate. Diversity of context beats raw frequency.

Kneser-Ney's insight is the best idea in classical language modelling, and it is worth stating as a principle: **what makes a word a good guess in an unfamiliar context is the variety of contexts it has appeared in, not how often it has appeared. Which is a statement about generalisation rather than about counting** — and it anticipates the distributional argument of Module 04 from inside the count-based paradigm.

THE REAL LIMIT IS THAT COUNTING CANNOT GENERALISE ACROSS SIMILAR WORDS

Having observed 'the cat sat on the mat' a thousand times tells an n-gram model precisely nothing about 'the dog sat on the rug' — the contexts are different strings, so the counts are entirely unrelated, and the model starts from zero.

Which is Module 01 §1's discreteness arriving as a concrete modelling failure — there is no notion of similarity between contexts, so every context must be observed directly, and the number of contexts grows exponentially with the order.

And smoothing does not fix this. Smoothing redistributes probability mass toward unseen events; it does not transfer what was learned about one word to a similar one — which is a different operation entirely, and no amount of back-off sophistication achieves it.

So the neural formulation's real advantage is parameter sharing: 'cat' and 'dog' receive nearby vectors, so evidence about one directly informs predictions about the other — which is **Module 04's** subject and the actual reason for the whole architecture, rather than depth or nonlinearity.

3 Perplexity

$$PP = \exp(-(1/N) \sum_i \log P(w_i \mid \text{context}))$$

- **The exponentiated average negative log likelihood per token**, computed on held-out text — so it is a direct function of how surprised the model was.
- **The interpretation is an effective branching factor: a perplexity of 100 means the model was as uncertain as if it were choosing uniformly among 100 options** at each step. Lower is better.
- **And it depends on the tokenisation** — so two models with different tokenisers are not comparable by perplexity at all (Module 02), since the same text is a different number of predictions. **This is the trap**, and it makes a good deal of published comparison harder to read than it looks.
- **Perplexity numbers are therefore comparable only within a fixed vocabulary and on a fixed test set** — which is why Project 1 requires all four models to be evaluated on the identical held-out split, and why that requirement is not pedantry.
- **It does not measure usefulness. A model with lower perplexity can be worse at your actual task**, and while the correlation is positive it is far from perfect — which is CSCE 633 Module 12's choose-the-metric-from-the-use argument.
- **It is dominated by the frequent, easy tokens** — function words, punctuation, and predictable continuations — so a large improvement there can mask no improvement at all on the content words you cared about. **This is the limitation that matters most**, because it means a perplexity gain can be entirely real and entirely irrelevant.
- **It is incomparable across tokenisers and across test sets**, as above — and test sets differ in difficulty far more than is usually acknowledged.
- **And it rewards hedging. A model that spreads probability broadly scores better than one that commits and is occasionally wrong** — which is sometimes exactly what you want and sometimes the opposite (Module 09 §2's decoding trade).
- **So use perplexity for development, where its cheapness and sensitivity are valuable, and report task metrics for any claim** (Module 10) — which is the division of labour that keeps it useful.

4 The neural formulation

1. look up an embedding for each context token
2. combine them somehow (concatenate, average, or a sequence model -- Modules 05 and 06)
3. a hidden layer, or several
4. a softmax over the vocabulary

WHAT THIS BUYS

similar words share parameters, so evidence transfers between them
 the context can be arbitrarily long, in principle
 and the model size does not grow with the number of observed contexts

WHAT IT COSTS

the softmax over a large vocabulary is the expensive part -- which is why sampled softmax, hierarchical softmax, and subword vocabularies (Module 02) all exist
 and it needs far more computation per prediction than a table lookup does

Parameter sharing between similar words is the whole gain — everything else in that architecture is machinery to make the sharing affordable, and keeping that straight prevents the common mistake of attributing the improvement to depth or to nonlinearity. **The original neural language model was shallow and still beat heavily tuned n-gram models**, because sharing was the thing that mattered.

AND THIS IS WHERE THE COURSE'S ARC BEGINS

Module 04 asks where the embeddings in step 1 come from, and finds that they can be learned from co-occurrence statistics alone — without any labels at all.

Module 05 asks how to combine a variable-length context in step 2, and finds that recurrence works in principle and forgets in practice.

Module 06 replaces recurrence with attention, which fixes the forgetting and parallelises the computation — two unrelated benefits from one change, which is why it won so decisively.

And Module 07 observes that the resulting model, trained at scale on this same free objective, turns out to be useful for essentially everything — which is where the field currently is, and is §1's callout vindicated at a scale nobody predicted.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Jurafsky & Martin, chapters 3 and 7 (free)	§1 through §4 , with the smoothing derivations and the perplexity discussion done properly.
Chen & Goodman — An Empirical Study of Smoothing Techniques (free)	§2's comparison, settled empirically — the study that established Kneser-Ney as the default.
Bengio et al. — A Neural Probabilistic Language Model (free)	§4 in the original , from 2003 — and the parameter sharing argument is made explicitly, which is why it is worth reading.
Jelinek — the history of statistical language modelling	§1 and §2 in context — how the field arrived at the next-token formulation, by somebody who was there.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write out the chain rule factorisation** and say which part is exact.
- 2 **Build a trigram model** on a corpus and count how many of its possible trigrams were observed.
- 3 **Evaluate it on held-out text** without smoothing, and report what happens.
- 4 **Add back-off, then interpolation, then Kneser-Ney**, and compare perplexities.
- 5 **Explain the Francisco example** in your own words.
- 6 **Demonstrate the generalisation failure**: train on 'cat' contexts and test on 'dog' ones.
- 7 **Compute perplexity with two different tokenisers** and show the numbers are incomparable.
- 8 **Measure perplexity on function words and content words separately**, and report the gap.
- 9 **Build a neural language model** with learned embeddings and compare on the same split.
- 10 **Attribute the improvement** to sharing rather than to depth, by ablating the hidden layer.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does a language model compute, and which part is exact?
- 2 Why does next-token prediction suffice for tasks it never saw?
- 3 What is the model actually optimised for, and what follows?
- 4 Why are most n-gram counts zero?
- 5 Name four smoothing methods and state Kneser-Ney's insight.
- 6 Why can counting not generalise, and why does smoothing not fix it?

- 7 Define perplexity and give its interpretation.
- 8 Why is perplexity incomparable across tokenisers?
- 9 Give four things perplexity does not measure, and the worst of them.
- 10 What is the whole gain of the neural formulation?